The lock is held until the thread exits, so care must be taken to avoid deadlock. It's recommended that the calling thread should exit as soon as possible and that no blocking operations be performed on this thread.

---

The user should ensure that waiting threads don't erroneously assume that the thread has exited when they are woken, particularly with the potential for spurious wakeups. This can be achieved by testing a predicate on the waiting thread that's only made `true` by the notifying thread under the protection of the mutex and without releasing the lock on the mutex prior to the call of `notify_all_at_thread_exit`.std::condition_variable_any class.

### D.2.2. std::condition_variable_any class

The `std::condition_variable_any` class allows a thread to wait for a condition to become `true`. Whereas `std::condition_variable` can be used only with `std::unique_lock<std::mutex>`, `std::condition_variable_any` can be used with *any* type that meets the `Lockable` requirements.

Instances of `std::condition_variable_any` aren't `CopyAssignable, Copy-Constructible, MoveAssignable`, or `MoveConstructible`.

*Class definition*

```
class condition_variable_any
{
public:
    condition_variable_any();
    ~condition_variable_any();

    condition_variable_any(
        condition_variable_any const& ) = delete;
    condition_variable_any& operator=(
        condition_variable_any const& ) = delete;
```